

VAPT REPORT : HACKED YOURSELF FIRST

A01 : Broken Access Control (IDOR)

Description :

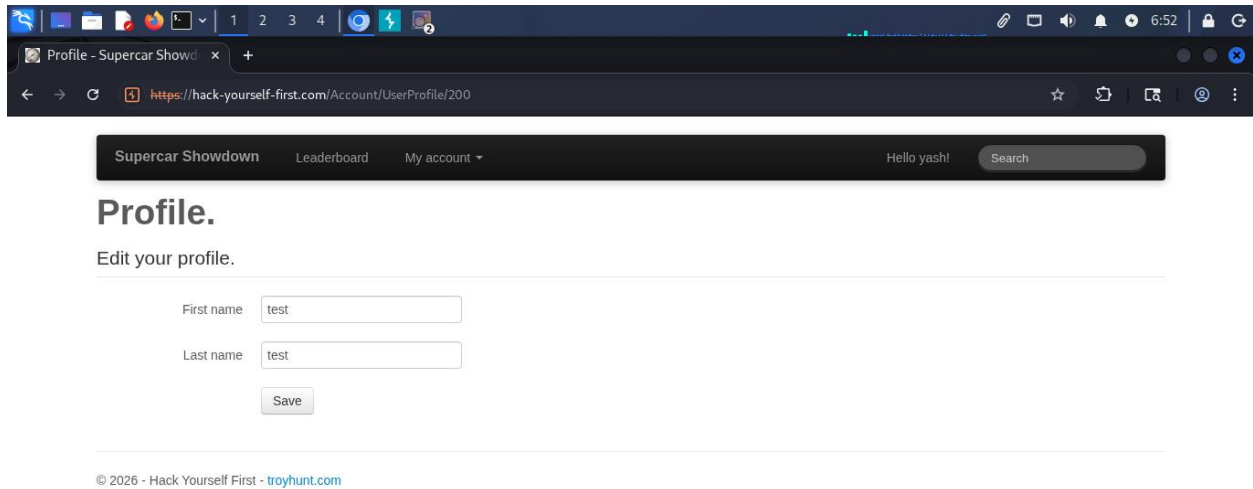
IDOR is a problem where the application shows internal IDs like user id and does not check properly if the user is allowed to see that data.

In this case, by changing the user id in the URL, it was possible to see other users data. This means proper checking was not done on server side.

Steps of attack :

1. Login into the application
2. Go to profile or account page
3. Observe the user id in URL
4. Change the id to another number
5. Reload the page and access other user data

PoC :



Impact :

- Can access other user data
- Privacy issue
- Data leakage possible
- Can lead to account takeover

Mitigation :

- Check access on server side
- Verify user before showing data
- Do not expose direct ids
- Use role based access control
- Test application regularly

A01 : Broken Access Control (Privilege Escalation)

Description :

Privilege escalation happens when a normal user becomes admin without permission.

In this case, cookie value `IsAdmin=false` was changed to `IsAdmin=true` and admin access was given. This shows server is trusting client side data.

Steps of attack :

1. Login as normal user
2. Open browser developer tools or proxy
3. Find cookie named `IsAdmin`
4. Change its value to `true`
5. Refresh and get admin access

PoC :

Burp Suite Community Edition v2026.2.3 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop

Request to https://hack-yourself-first.com:443 [104.21.89.41] Open browser ?

Time	Type	Direction	Method	URL	Status code	Length
07:26:27.13 Ap...	HTTP	→ Request	GET	https://hack-yourself-first.com/bundles/jqueryval?v=k095ZRLUEvNzbFwaTlhaJ0t0ngQ4k32HedumCbRM1		
07:26:27.13 Ap...	HTTP	→ Request	GET	https://hack-yourself-first.com/		
07:26:27.13 Ap...	HTTP	→ Request	GET	https://hack-yourself-first.com/bundles/bootstrap?v=NE-CTK4A7Gr22gKpUJ559z6HOS1t1Zd8jgam_8c3l0t		
07:26:27.13 Ap...	HTTP	→ Request	GET	https://hack-yourself-first.com/bundles/jquery?v=yMmPMITxexYcoWICWW3JYgHOf9kAasOfb-W5l00lA1		

Request

Pretty Raw Hex

```
1 GET /bundles/jqueryval?v=k095ZRLUEvNzbFwaTlhaJ0t0ngQ4k32HedumCbRM1 HTTP/2
2 Host: hack-yourself-first.com
3 Cookie: ASP.NET_SessionId=k3pgfpevqeaobongoxdedrav; VisitStart=4/13/2026 10:18:58 AM; IsAdmin=true; AuthCookie=EA07AGB5C02950A5097457809B84FBC0C9895A71F382ABE45C2063AB050081624B9B480BEDC2AB909C82A9B454E024AC2DF674F60FB68979CC120E1090E89BCAF314F0E31A3F91156A1E72C72CE965EBAAGFFSF0584E4C40581E49E29031C672090CE25F43DB91BD9363C80767670
4 Sec-Purpose: prefetch
5 Sec-Fetch-Site: same-origin
6 Sec-Fetch-Mode: no-cors
7 Sec-Fetch-Dest: script
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
9 Accept-Encoding: gzip, deflate, br
10 Accept-Language: en-US,en;q=0.9
11 Priority: u=4, i
12
13
```

Inspector

Request attributes 2

Request query parameters 1

Request body parameters 0

Request cookies 4

Request headers 16

Event log (10) All issues Memory: 268.1MB of 2.35GB Disabled

Supercar Showdown - Super

https://hack-yourself-first.com

Supercar Showdown Leaderboard My account Admin Hello yo! Search

Manage users

Ferrari LaFerrari

The LaFerrari is the first hybrid from Ferrari, providing the highest power output of any Ferrari whilst decreasing fuel consumption by 40 percent.

View the LaFerrari

Top manufacturers

Who's the best of the best

https://hack-yourself-first.com/Secret/Users



Users - Supercar Showdown | Leaderboard | My account | Hello yo! | Search

Users

Email	First name	Last name	Password
troyhunt@hotmail.com	NULL	USER_edited	123456789
sebastianvettel@f1.com	hacker1	hacker	sunshine
kimiraikkonen@f1.com	Kimi	Räikkönen	iloveyou
fernandoalonso@f1.com	Fernando	Alonso	11111111
lewishamilton@f1.com	Lewis	Hamilton	thx1138
felipemassa@f1.com	Felipe	Massa	rainbow
markwebber@f1.com	Mark	Webber	gogogo
romaingrosjean@f1.com	Romain	Grosjean	scorpion
pauldiresta@f1.com	Paul	di Resta	jordan23
nicorosberg@f1.com	Nico	Rosberg	trinity
jensonbutton@f1.com	Jenson	Button	wwwwww
sergioperez@f1.com	Sergio	Pérez	america1
danielricciardo@f1.com	Daniel	Ricciardo	millions

Impact :

- Unauthorized admin access
- Full control over system
- Data can be changed or deleted
- System can be compromised

Mitigation :

- Always check roles on server side
- Do not trust cookies
- Implement role based access control
- Store roles securely
- Validate every request

A02 : Security Misconfiguration

Description :

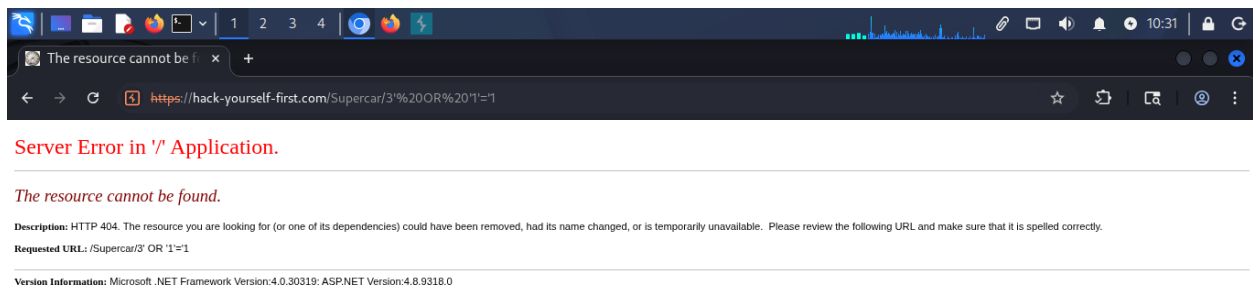
This happens when system is not configured properly.

Here, when wrong input was given, detailed error was shown which revealed database and backend info like ASP.NET and SQL Server.

Steps of attack :

1. Enter invalid input in form
2. Submit request
3. Observe detailed error message
4. Collect system information

PoC :



Impact :

- Sensitive information exposed

- Helps attacker understand system
- Can help in further attacks
- Shows internal details

Mitigation :

- Disable detailed errors
- Use custom error pages
- Log errors on server
- Hide system details
- Review configurations

A03 : Software Supply Chain Failures

Description :

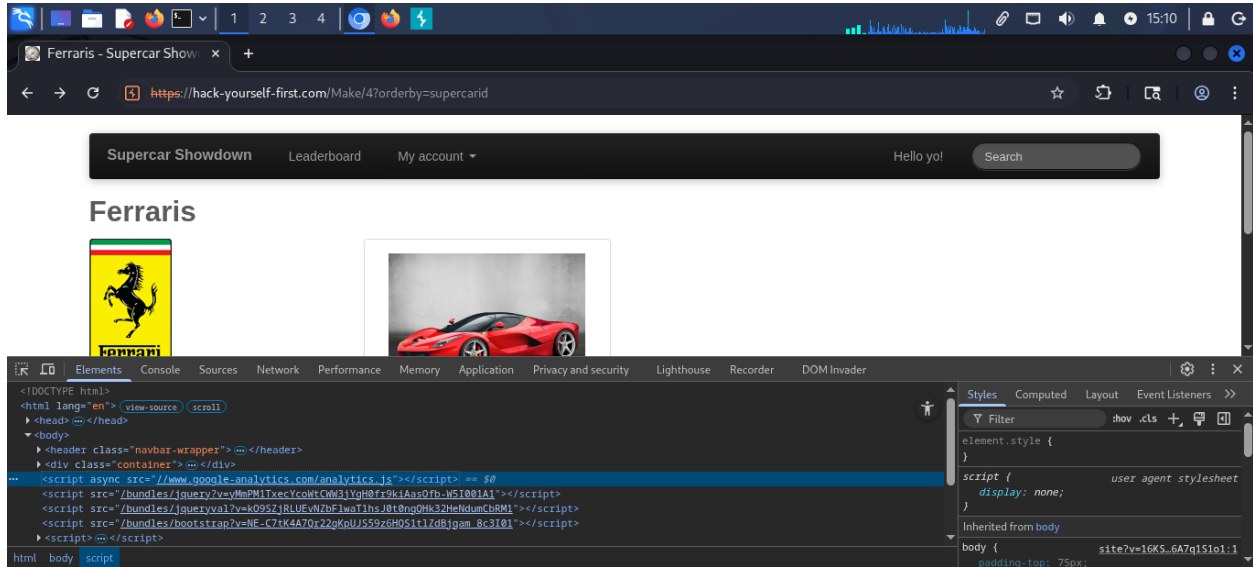
This happens when external libraries are used without checking security.

In this app, scripts like jQuery and Bootstrap are used without SRI, so integrity is not verified.

Steps of attack :

1. Check page source
2. Find external scripts
3. See if SRI is missing
4. Replace script with malicious one

PoC :



Impact :

- Malicious code execution
- Full application compromise
- Data theft possible
- Session stealing

Mitigation :

- Use SRI for scripts
- Use trusted libraries
- Update dependencies
- Avoid unknown sources
- Scan for vulnerabilities

A04 : Cryptographic Failures

Description :

Sensitive data is not protected properly.

Here, application uses HTTP instead of HTTPS, so data like username and password can be captured.

Steps of attack :

1. Start proxy tool
2. Login into application
3. Capture request
4. See data in plain text

PoC :

The screenshot displays the Burp Suite interface with the 'Proxy' tab selected. A list of intercepted requests is shown, with the third request (06:19:55) selected. The 'Request' tab is active, showing the raw HTTP request. The request body contains the following data in plain text:

```
Host: hack-yourself-first.com
Cookie: ASP.NET_SessionId=kx3pgfpvqeaabomgoxdndraw; VisitStart=4/13/2026 10:18:58 AM
Content-Length: 83
Cache-Control: max-age=0
Sec-Ch-Ua: "Chrome", v="145", "Not:A-Brand", v="99"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Linux"
Accept-Language: en-US,en;q=0.9
Origin: https://hack-yourself-first.com
Content-Type: application/x-www-form-urlencoded
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Referer: https://hack-yourself-first.com/Account/Register
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
Email=yash40@gmail.com&FirstName=yash&LastName=d&Password=yash&ConfirmPassword=yash
```

The 'Inspector' panel on the right shows the request attributes, query parameters, body parameters, cookies, and headers. The status bar at the bottom indicates 0 highlights and a memory usage of 180.8MB of 2.35GB.

Impact :

- Password leakage
- MITM attack possible
- Session hijacking
- Loss of privacy

Mitigation :

- Use HTTPS everywhere
- Use TLS 1.2 or higher
- Redirect HTTP to HTTPS
- Secure cookies
- Do not send plain data

A05 : Injection (XSS)

Description :

XSS happens when user input is not filtered and script runs in browser.

Here, script

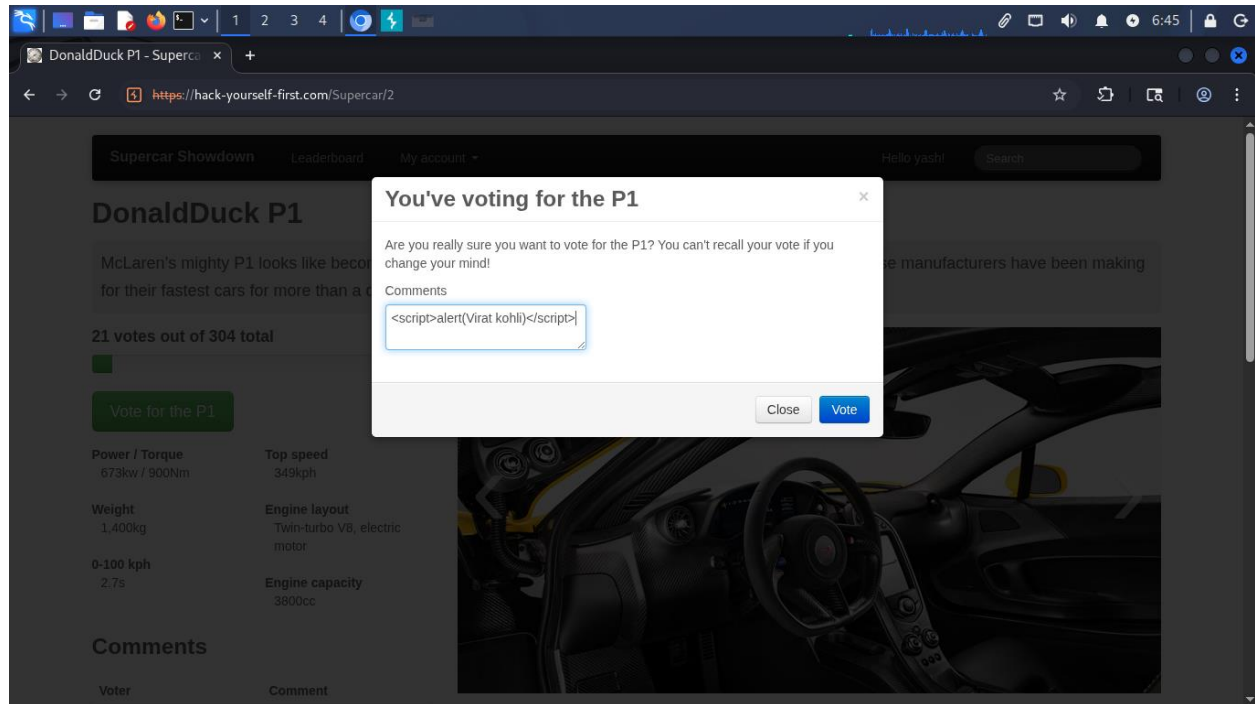
was executed.

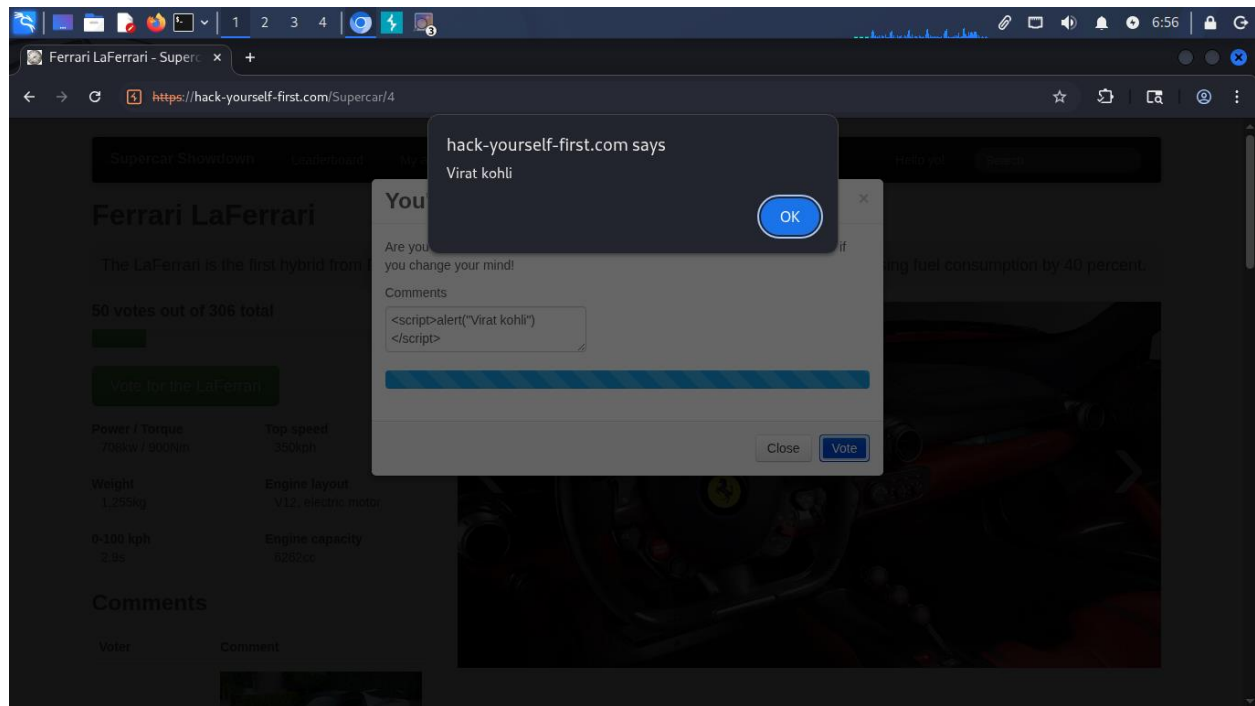
Steps of attack :

1. Find input field
2. Enter script payload
3. Submit form

4. Script executes in browser

PoC :





Impact :

- Script execution
- Cookie theft
- Session hijacking
- Page defacement
- Redirect to malicious site

Mitigation :

- Validate input
- Encode output
- Use CSP
- Avoid direct rendering

- Use secure frameworks

A06 : Insecure Design

Description :

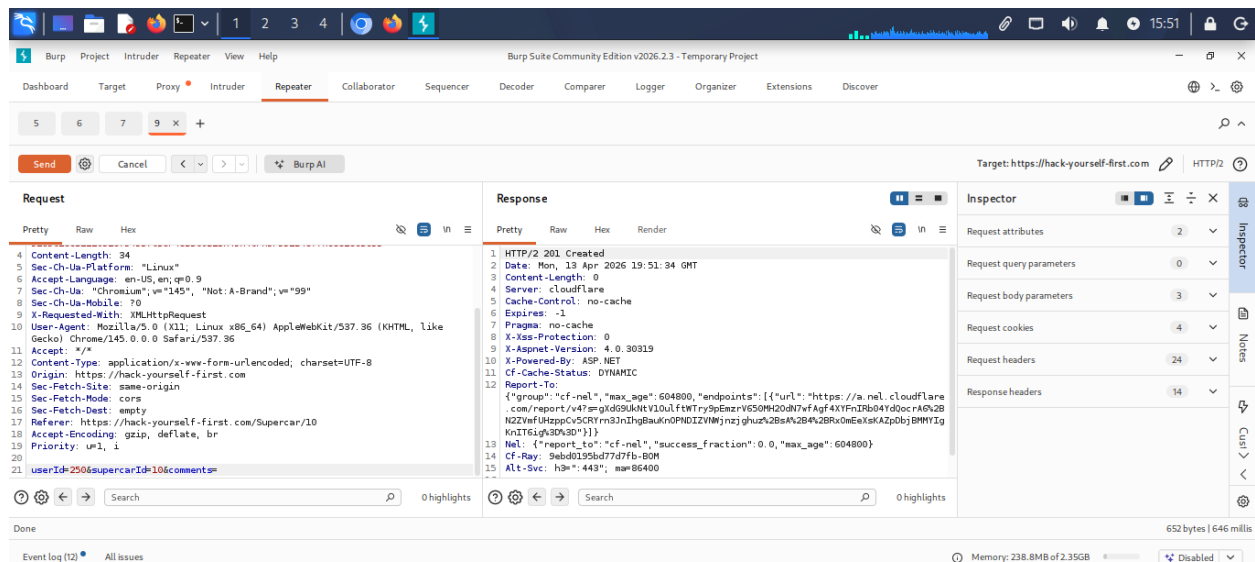
This is a design issue where proper logic is not applied.

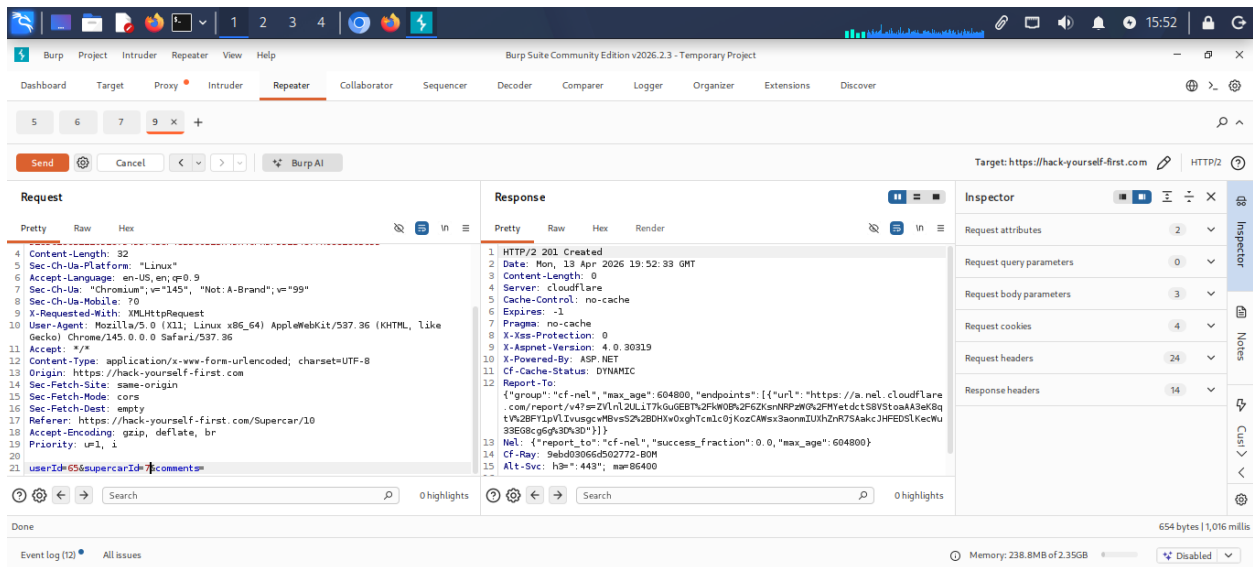
Here, voting system allows multiple votes from same user.

Steps of attack :

1. Vote once
2. Capture request
3. Send request multiple times
4. Votes increase

PoC :





Impact :

- Manipulation of results
- Data integrity loss
- Unfair system
- Feature abuse

Mitigation :

- Validate logic on server
- Allow one vote per user
- Check duplicate requests
- Use rate limiting
- Secure design

A07 : Authentication Failures

Description :

This happens when login and session is not handled properly.

Here, no lockout on multiple attempts and session id not changed after login.

Steps of attack :

1. Try multiple passwords
2. No restriction observed
3. Fix session id
4. Login and reuse session

PoC :

The screenshot shows the Burp Suite interface during an intruder attack on <https://hack-yourself-first.com>. The 'Results' tab is active, showing a table of requests and responses. The final request (index 5) is highlighted, showing a successful login with a 302 status code. The response (index 5) is also highlighted, showing a JSON token and session information.

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		302	1157			1748	
1	123456	200	381			6891	
2	password	200	375			6889	
3	admin	200	434			6891	
4	12345678	200	631			6895	
5	yo	302	342			1758	

Request

```
1 POST /Account/Login HTTP/2
2 Host: hack-yourself-first.com
3 Cookie: ASP.NET_SessionId=wuvqzgejo4zhyvv4ug0drav0; VisitStart=4/13/2026 2:14:07 PM; IsAdmin=False
4 Content-Length: 48
5 Cache-Control: max-age=0
6 Sec-Ch-Ua: "Chromium"; v="145", "Not:A-Brand"; v="99"
7 Sec-Ch-Ua-Mobile: ?0
```

Response

```
24 {
25   "version": "2024.11.0",
26   "token": "8fcd3bdc2f941d3be866c75e6b1500",
27   "p": 1,
28   "server_timing": {
29     "name": {
30       "cfCacheStatus": true,
31       "cfEdge": true,
32       "cfExtPri": true,
33       "cf4": true,
34       "cfOrigin": true,
35       "cfSpeedBrain": true,
36       "location_startswith": null
37     }
38   }
39 }
40 </script>
41 </body>
42 </html>
```

Inspector

- Request attributes: 2
- Request body parameters: 3
- Request cookies: 3
- Request headers: 26
- Response headers: 18

Impact :

- Brute force possible
- Account takeover
- Session hijacking
- Weak authentication

Mitigation :

- Add rate limit
- Lock account after attempts
- Change session id after login
- Use strong passwords
- Add MFA

A08 : Software / Data Integrity Failures

Description :

This happens when application trusts user input.

Here, parameters like userId were modified and accepted.

Steps of attack :

1. Capture request
2. Modify parameters
3. Send request again
4. Server accepts changes

PoC :

The screenshot shows the Burp Suite Repeater interface. The target is `https://hack-yourself-first.com`. The request is an `HTTP/2 POST` to `/api/v1/users`. The request body is a JSON object: `{\"user\": \"super\", \"password\": \"123456\", \"comment\": \"hacker\"}`. The response is an `HTTP/2 201 Created` status with a `Content-Type: application/json` header. The response body is a JSON object: `{\"id\": 1, \"username\": \"super\", \"password\": \"123456\", \"comment\": \"hacker\", \"created_at\": \"2023-04-13T19:51:34.346Z\"}`.

The screenshot shows the Burp Suite Repeater interface. The target is `https://hack-yourself-first.com`. The request is an `HTTP/2 POST` to `/api/v1/users`. The request body is a JSON object: `{\"user\": \"super\", \"password\": \"123456\", \"comment\": \"hacker\"}`. The response is an `HTTP/2 201 Created` status with a `Content-Type: application/json` header. The response body is a JSON object: `{\"id\": 1, \"username\": \"super\", \"password\": \"123456\", \"comment\": \"hacker\", \"created_at\": \"2023-04-13T19:52:33.046Z\"}`.

Impact :

- Data manipulation
- Business logic bypass
- Unauthorized changes
- Integrity loss

Mitigation :

- Validate on server
- Do not trust client data
- Add authorization checks
- Protect sensitive operations
- Use secure tokens

A09 : Security Logging & Monitoring Failures

Description :

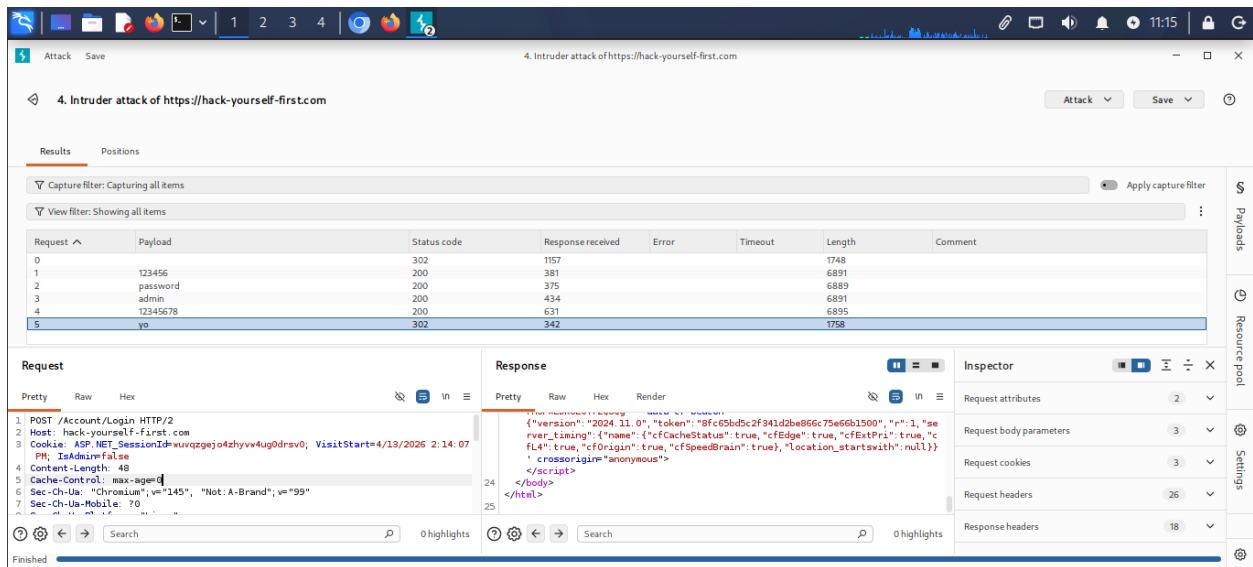
Application does not track or detect attacks.

Here, multiple login attempts were not blocked or logged.

Steps of attack :

1. Perform repeated login attempts
2. Observe no blocking
3. Try multiple actions
4. No alerts generated

PoC :



Impact :

- Attacks not detected
- Delay in response
- Account compromise
- No evidence

Mitigation :

- Enable logging
- Monitor activities
- Add alerts
- Use SIEM tools
- Apply rate limit

A10 : Improper Exception Handling

Description :

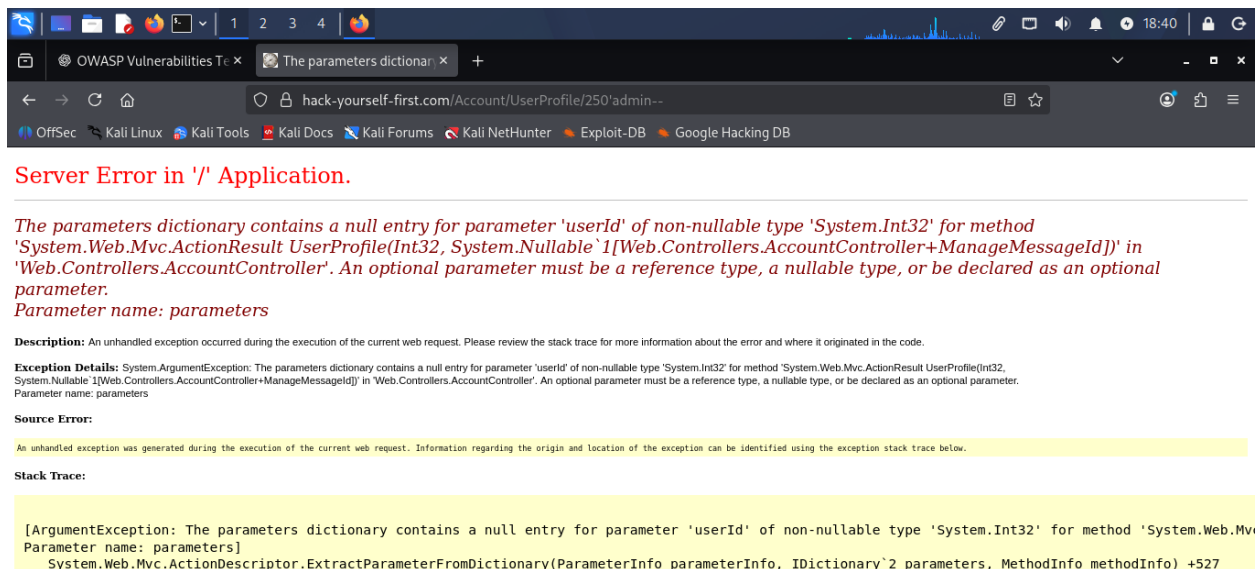
Application shows detailed error when exception occurs.

Here, input like 'admin-- shows ASP.NET error with internal details.

Steps of attack :

1. Enter invalid input
2. Submit form
3. Observe detailed error
4. Collect system info

PoC :



Server Error in '/' Application.

*The parameters dictionary contains a null entry for parameter 'userId' of non-nullable type 'System.Int32' for method 'System.Web.Mvc.ActionResult UserProfile(Int32, System.Nullable`1[Web.Controllers.AccountController+ManageMessageId])' in 'Web.Controllers.AccountController'. An optional parameter must be a reference type, a nullable type, or be declared as an optional parameter.
Parameter name: parameters*

Description: An unhandled exception occurred during the execution of the current web request. Please review the stack trace for more information about the error and where it originated in the code.

Exception Details: System.ArgumentException: The parameters dictionary contains a null entry for parameter 'userId' of non-nullable type 'System.Int32' for method 'System.Web.Mvc.ActionResult UserProfile(Int32, System.Nullable`1[Web.Controllers.AccountController+ManageMessageId])' in 'Web.Controllers.AccountController'. An optional parameter must be a reference type, a nullable type, or be declared as an optional parameter.
Parameter name: parameters

Source Error:

An unhandled exception was generated during the execution of the current web request. Information regarding the origin and location of the exception can be identified using the exception stack trace below.

Stack Trace:

```
[ArgumentException: The parameters dictionary contains a null entry for parameter 'userId' of non-nullable type 'System.Int32' for method 'System.Web.Mvc.ActionResult UserProfile(Int32, System.Nullable`1[Web.Controllers.AccountController+ManageMessageId])' in 'Web.Controllers.AccountController'. An optional parameter must be a reference type, a nullable type, or be declared as an optional parameter.  
Parameter name: parameters]  
   System.Web.Mvc.ActionDescriptor.ExtractParameterFromDictionary(ParameterInfo parameterInfo, IDictionary`2 parameters, MethodInfo methodInfo) +527
```

Impact :

- Internal details exposed
- Helps attacker
- Targeted attacks possible
- Bad user experience

Mitigation :

- Show generic errors
- Disable debug mode
- Validate inputs
- Handle exceptions properly
- Do not expose system details

A07 : Authentication Failures (Session ID Reuse)

Description :

This issue happens when the application does not generate a unique session id for each user session.

During testing, it was observed using Burp Suite that even though authentication cookies were different for each user, the session id remained the same (g4ajpsr0yuzwiedm2igydzxl).

This shows that the application is not handling session management properly and is reusing the same session id across multiple users.

Steps of attack :

Login into the application with first user

Intercept request using Burp Suite

Note the session id value

Logout and login with another user

Capture the request again

Observe that session id is same but authentication cookie is different

PoC :

Same session id observed for different users: g4ajpsr0yuzwiedm2igydzxl

The first screenshot shows a GET request to `/Account/Login` with the following cookies:

```
Cookie: ASP.NET_SessionId=g4ajpsr0yuzwiedm2igydzxl; VisitStart=5/4/2026 3:50:12 PM; IsAdmin=true; Sec-Ch-Ua-\"Chromium\";v=\"145\", \"Not:A-Brand\";v=\"99\"; Sec-Ch-Ua-Mobile: 70; Sec-Ch-Ua-Platform: \"Linux\"; Accept-Language: en-US,en;q=0.9; Upgrade-Insecure-Requests: 1; User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
```

The second screenshot shows a POST request to `/Account/Login` with the following cookies:

```
Cookie: ASP.NET_SessionId=g4ajpsr0yuzwiedm2igydzxl; VisitStart=5/4/2026 3:50:12 PM; IsAdmin=false; Content-Length: 46; Cache-Control: max-age=0; Sec-Ch-Ua-\"Chromium\";v=\"145\", \"Not:A-Brand\";v=\"99\"; Sec-Ch-Ua-Mobile: 70; Sec-Ch-Ua-Platform: \"Linux\"; Accept-Language: en-US,en;q=0.9; Origin: https://hack-yourself-first.com; Content-Type: application/x-www-form-urlencoded; Upgrade-Insecure-Requests: 1; User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36; Sec-Fetch-Site: same-origin
```

Both requests show the same session ID in the response cookies, indicating a session fixation attack.

Impact :

Session fixation or hijacking possible

Multiple users share same session context

Unauthorized access to other accounts

Sensitive data exposure

Weak session management

Mitigation :

Generate unique session id for every user session

Regenerate session id after login

Bind session id with user account on server side

Invalidate old sessions properly

Use secure session handling mechanisms

Conclusion :

In this testing, 12 vulnerabilities were found in the application. These issues show weak security in areas like access control, authentication, and input validation. Some vulnerabilities can lead to unauthorized access and data leakage. The application needs proper security improvements like validation, secure session handling, and regular testing to make it safe.